Non-Patent Literature: Source Code for Prototype Implementation

```
# -*- coding: utf-8 -*-
"""
Finds the first opportunity to bounce a signal off a supported LEO, MEO,
or GEO satellite between two points on Earth.

Intended Use: Forwards cat videos (or Kats, or Lyons videos) to a friend on another
continent with the highest possible latency, using a multi-billion dollar satellite
constellation as a mirror. Please do not use for practical purposes.
Or for communicating with aliens. They have better FTL methods.

Date: June 25, 2025
"""
from flask import Flask, render_template, request, jsonify, Response
from datetime import datetime, timedelta, timezone
import numpy as np
import requests
import io
import os
import time
import traceback
import uuid
import json

from skyfield.api import load, wgs84
```

```
from google.cloud import firestore, tasks_v2

# --- App Initialization ---
app = Flask(__name__)

# --- Initialize clients only if in production ---
db = None
tasks_client = None
if os.environ.get('GAE_ENV', '').startswith('standard'):
    try:
        db = firestore.Client(database='bounce')
        tasks_client = tasks_v2.CloudTasksClient()
        print("Successfully connected to Firestore and Cloud Tasks."
            )
    except Exception as e:
        print(f"CRITICAL ERROR: Failed to initialize Google Cloud
            clients: {e}")

# --- Environment Config & Constants ---
GCP_PROJECT_ID = os.environ.get('GCP_PROJECT_ID')
GCP_QUEUE_NAME = os.environ.get('GCP_QUEUE_NAME')
GCP_QUEUE_LOCATION = os.environ.get('GCP_QUEUE_LOCATION')
WORKER_URL = os.environ.get('WORKER_URL')

timescale = load.timescale()
TLE_CACHE_DIR = "/tmp/tle_cache/"
TLE_CACHE_DURATION_SEC = 7200 # Cache for 2 hours
TLE_URLS_LEO = {
    "LEO": {
```

```
52          "Starlink": "https://celestrak.org/NORAD/elements/gp.php?
                GROUP=starlink&FORMAT=tle",
53          "OneWeb": "https://celestrak.org/NORAD/elements/gp.php?GROUP
                =oneweb&FORMAT=tle",
54          "Kuiper": "https://celestrak.org/NORAD/elements/gp.php?GROUP
                =kuiper&FORMAT=tle",
55      }
56  }
57  TLE_URLS_MEO_GEO = {
58      "MEO/GEO": {
59          "GPS": "https://celestrak.org/NORAD/elements/gp.php?GROUP=
                gps-ops&FORMAT=tle",
60          "Galileo": "https://celestrak.org/NORAD/elements/gp.php?
                GROUP=galileo&FORMAT=tle",
61          "Geostationary": "https://celestrak.org/NORAD/elements/gp.
                php?GROUP=geo&FORMAT=tle",
62      }
63  }
64  LEO_MAX_DISTANCE_KM = 5000
65  MOVABLE_SEARCH_DURATION_SEC = 600
66  FIXED_SEARCH_DURATION_SEC = 3600
67  SEARCH_STEP_SEC = 30
68  BROAD_PHASE_RADIUS_M = 3000 * 1000
69  MOVABLE_MIN_ELEVATION_DEG = 10.0
70  MOVABLE_TOLERANCE_DEG = 10.0
71  FIXED_ELEVATION_DEG = 25.0
72  KEYHOLE_TOLERANCE_DEG = 2.5
73
74  ANTENNA_CONFIGS = [
```

```
75      {'direction': 'N', 'azimuth': 0.0, 'elevation':
            FIXED_ELEVATION_DEG},
76      {'direction': 'E', 'azimuth': 90.0, 'elevation':
            FIXED_ELEVATION_DEG},
77      {'direction': 'S', 'azimuth': 180.0, 'elevation':
            FIXED_ELEVATION_DEG},
78      {'direction': 'W', 'azimuth': 270.0, 'elevation':
            FIXED_ELEVATION_DEG},
79  ]
80
81  # --- Helper Functions ---
82  def get_tle_data(url_map):
83      if not os.path.exists(TLE_CACHE_DIR): os.makedirs(TLE_CACHE_DIR)
84      full_tle_string = ""
85      headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64;
            x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome
            /58.0.3029.110 Safari/537.36'}
86      for group_name, urls in url_map.items():
87          for name, url in urls.items():
88              cache_file_path = os.path.join(TLE_CACHE_DIR, f"{name.
                    replace('/', '_')}.txt")
89              if os.path.exists(cache_file_path) and (time.time() - os
                    .path.getmtime(cache_file_path)) <
                    TLE_CACHE_DURATION_SEC:
90                  with open(cache_file_path, 'r') as f:
                        full_tle_string += f.read() + "\n"
91                  continue
92              try:
93                  response = requests.get(url, timeout=15, headers=
```

```
                    headers)
94                  response.raise_for_status()
95                  tle_data = response.text
96                  with open(cache_file_path, 'w') as f: f.write(
                        tle_data)
97                  full_tle_string += tle_data + "\n"
98              except requests.exceptions.RequestException as e:
99                  print(f"Warning: Failed to fetch from {url}: {e}")
100                 if os.path.exists(cache_file_path):
101                     with open(cache_file_path, 'r') as f:
                            full_tle_string += f.read() + "\n"
102     return full_tle_string if full_tle_string else None
103
104 def parse_tle_string(tle_data):
105     try:
106         temp_tle_filename = '/tmp/temp_tle_file.txt'
107         with open(temp_tle_filename, 'w') as f: f.write(tle_data)
108         satellites = load.tle_file(temp_tle_filename)
109         name_map = {sat.model.satnum: sat.name for sat in satellites
                }
110         return satellites, name_map
111     finally:
112         if os.path.exists(temp_tle_filename): os.remove(
                temp_tle_filename)
113
114 def haversine_distance(lat1, lon1, lat2, lon2):
115     R = 6371
116     dLat, dLon, lat1_rad, lat2_rad = map(np.radians, [lat2 - lat1,
          lon2 - lon1, lat1, lat2])
```

```
117     a = np.sin(dLat/2)**2 + np.cos(lat1_rad) * np.cos(lat2_rad) * np
            .sin(dLon/2)**2
118     c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1-a))
119     return R * c
120
121 def get_los_vector(lat, lon, az, el):
122     lat_rad, lon_rad, el_rad, az_rad = map(np.radians, [lat, lon, el
            , az])
123     s_el, c_el, s_az, c_az = np.sin(el_rad), np.cos(el_rad), np.sin(
            az_rad), np.cos(az_rad)
124     enu_vec = np.array([s_az * c_el, c_az * c_el, s_el])
125     rot_matrix = np.array([
126         [-np.sin(lon_rad), -np.cos(lon_rad)*np.sin(lat_rad), np.cos(
                lon_rad)*np.cos(lat_rad)],
127         [np.cos(lon_rad), -np.sin(lon_rad)*np.sin(lat_rad), np.sin(
                lon_rad)*np.cos(lat_rad)],
128         [0, np.cos(lat_rad), np.sin(lat_rad)]
129     ])
130     return rot_matrix.dot(enu_vec)
131
132 # --- THE TWO SEARCH ENGINES (Now check only a single timestamp) ---
133 def find_movable_bounce_at_time(satellites, name_map, sender,
        recipient, t):
134     sender_geo, recipient_geo = wgs84.latlon(sender['lat'], sender['
            lon']), wgs84.latlon(recipient['lat'], recipient['lon'])
135     use_kinematic_filter = abs(recipient['lat'] - sender['lat']) >=
            1.0
136     if use_kinematic_filter: required_direction_is_ascending = (
            recipient['lat'] > sender['lat'])
```

```
137
138     p_sender_obj, p_recipient_obj = sender_geo.at(t), recipient_geo.
           at(t)
139     p_sender_m, p_recipient_m = p_sender_obj.position.m,
           p_recipient_obj.position.m
140     midpoint_ecef = (p_sender_m + p_recipient_m) / 2
141
142     initial_candidates = satellites
143     if use_kinematic_filter: initial_candidates = [sat for sat in
           satellites if (sat.at(t).velocity.km_per_s[2] > 0) ==
           required_direction_is_ascending]
144
145     nearby_candidates = [(sat, sat.at(t)) for sat in
           initial_candidates if np.linalg.norm(sat.at(t).position.m -
           midpoint_ecef) < BROAD_PHASE_RADIUS_M]
146
147     for sat, p_sat_obj in nearby_candidates:
148         p_sat_m = p_sat_obj.position.m
149         v_normal, v_in, v_out = p_sat_m/np.linalg.norm(p_sat_m),
               p_sender_m - p_sat_m, p_recipient_m - p_sat_m
150         v_in_norm, v_out_norm = v_in / np.linalg.norm(v_in), v_out /
                np.linalg.norm(v_out)
151         if not np.isclose(np.dot(v_in_norm, v_normal), np.dot(
               v_out_norm, v_normal), atol=MOVABLE_TOLERANCE_DEG/10):
               continue
152         elevation_sender, azimuth_sender, _ = (p_sat_obj -
               p_sender_obj).altaz()
153         if elevation_sender.degrees < MOVABLE_MIN_ELEVATION_DEG:
               continue
```

```
154             elevation_recipient, azimuth_recipient, _ = (p_sat_obj -
                    p_recipient_obj).altaz()
155             if elevation_recipient.degrees < MOVABLE_MIN_ELEVATION_DEG:
                    continue
156             sat_geo = wgs84.geographic_position_of(p_sat_obj)
157             return {
158                 'satellite_name': name_map.get(sat.model.satnum, f"ID {
                        sat.model.satnum}"), 'satellite_norad_id': sat.model.
                        satnum,
159                 'event_timestamp_utc': t.utc_iso(),
160                 'sender_location': [sender['lat'], sender['lon']], '
                        recipient_location': [recipient['lat'], recipient['
                        lon']],
161                 'satellite_ground_track': [sat_geo.latitude.degrees,
                        sat_geo.longitude.degrees], 'satellite_altitude_km':
                        sat_geo.elevation.km,
162                 'elevation_sender': elevation_sender.degrees, '
                        azimuth_sender': azimuth_sender.degrees,
163                 'elevation_recipient': elevation_recipient.degrees, '
                        azimuth_recipient': azimuth_recipient.degrees,
164                 'stats': {'total': len(satellites), 'kinematic': len(
                        initial_candidates), 'nearby': len(nearby_candidates)
                        }
165             }
166         return None
167
168 def find_fixed_bounce_at_time(satellites, name_map, sender,
        recipient, t, plausible_paths):
169         sender_geo, recipient_geo = wgs84.latlon(sender['lat'], sender['
```

```
        lon']), wgs84.latlon(recipient['lat'], recipient['lon'])
170     p_sender_m, p_recipient_m = sender_geo.at(t).position.m,
        recipient_geo.at(t).position.m
171
172     for sender_antenna, recipient_antenna in plausible_paths:
173         v_los_sender = get_los_vector(sender['lat'], sender['lon'],
            sender_antenna['azimuth'], sender_antenna['elevation'])
174         v_los_recipient = get_los_vector(recipient['lat'], recipient
            ['lon'], recipient_antenna['azimuth'], recipient_antenna[
            'elevation'])
175         for sat in satellites:
176             p_sat_obj, p_sat_m = sat.at(t), sat.at(t).position.m
177             v_sender_to_sat_norm = (p_sat_m - p_sender_m) / np.
                linalg.norm(p_sat_m - p_sender_m)
178             angle_diff = np.degrees(np.arccos(np.clip(np.dot(
                v_sender_to_sat_norm, v_los_sender), -1.0, 1.0)))
179             if angle_diff < KEYHOLE_TOLERANCE_DEG:
180                 v_normal = p_sat_m / np.linalg.norm(p_sat_m)
181                 v_in_norm = -v_sender_to_sat_norm
182                 v_reflection = v_in_norm - 2 * np.dot(v_in_norm,
                    v_normal) * v_normal
183                 v_sat_to_recipient_norm = (p_recipient_m - p_sat_m)
                    / np.linalg.norm(p_recipient_m - p_sat_m)
184                 reflection_angle_diff = np.degrees(np.arccos(np.clip
                    (np.dot(v_reflection, v_sat_to_recipient_norm),
                    -1.0, 1.0)))
185                 if reflection_angle_diff < KEYHOLE_TOLERANCE_DEG:
186                     angle_diff_recipient = np.degrees(np.arccos(np.
                        clip(np.dot(-v_sat_to_recipient_norm,
```

```
                        v_los_recipient), -1.0, 1.0)))
187                     if angle_diff_recipient < KEYHOLE_TOLERANCE_DEG:
188                         sat_geo = wgs84.geographic_position_of(
                                p_sat_obj)
189                         return {
190                             'sender_antenna_direction':
                                    sender_antenna['direction'], '
                                    recipient_antenna_direction':
                                    recipient_antenna['direction'],
191                             'satellite_name': name_map.get(sat.model
                                    .satnum, f"ID {sat.model.satnum}"),
192                             'satellite_norad_id': sat.model.satnum,
                                    'event_timestamp_utc': t.utc_iso(),
193                             'satellite_ground_track': [sat_geo.
                                    latitude.degrees, sat_geo.longitude.
                                    degrees], 'satellite_altitude_km':
                                    sat_geo.elevation.km,
194                             'stats': {}
195                         }
196     return None
197
198 # --- MAIN ROUTER and Asynchronous Handlers ---
199 @app.route('/')
200 def index():
201     return render_template('index.html')
202
203 @app.route('/find-bounce', methods=['POST'])
204 def find_bounce_router():
205     data = request.json
```

```
206     try:
207         sender, recipient, mode = data.get('sender'), data.get('
                recipient'), data.get('mode', 'movable')
208         if not all([sender, recipient, mode]): return jsonify({'
                status': 'ERROR', 'message': 'Invalid input'}), 400
209 
210         search_id = str(uuid.uuid4())
211         search_duration = MOVABLE_SEARCH_DURATION_SEC if mode == '
                movable' else FIXED_SEARCH_DURATION_SEC
212         total_steps = search_duration // SEARCH_STEP_SEC
213 
214         db.collection('hyper_searches').document(search_id).set({
215             'status': 'PENDING', 'created_at': firestore.
                    SERVER_TIMESTAMP,
216             'results': [], 'steps_completed': 0, 'total_steps':
                    total_steps, 'mode': mode
217         })
218 
219         parent = tasks_client.queue_path(GCP_PROJECT_ID,
                GCP_QUEUE_LOCATION, GCP_QUEUE_NAME)
220         t_search_start_iso = datetime.now(timezone.utc).isoformat()
221 
222         for t_offset in range(0, search_duration, SEARCH_STEP_SEC):
223             task_payload = {
224                 'search_id': search_id, 'sender': sender, 'recipient
                        ': recipient,
225                 'mode': mode, 't_search_start_iso':
                        t_search_start_iso, 't_offset': t_offset
226             }
```

```
227             task = {'http_request': {'http_method': 'POST', 'url': f
                    "{WORKER_URL}/run-timeslice-worker", 'headers': {'
                    Content-Type': 'application/json'}, 'body': json.
                    dumps(task_payload).encode() }, 'dispatch_deadline':
                    timedelta(minutes=10) }
228             tasks_client.create_task(parent=parent, task=task)
229
230         print(f"Successfully created {total_steps} parallel worker
                tasks for search {search_id}.")
231         return jsonify({'status': 'Search initiated', 'search_id':
                search_id})
232     except Exception as e:
233         traceback.print_exc()
234         return jsonify({'status': 'ERROR', 'message': str(e)}), 500
235
236 @app.route('/run-timeslice-worker', methods=['POST'])
237 def run_timeslice_worker():
238     data = request.json
239     search_id, sender, recipient, mode, t_search_start_iso, t_offset
            = \
240         data.get('search_id'), data.get('sender'), data.get('
                recipient'), \
241         data.get('mode'), data.get('t_search_start_iso'), data.get('
                t_offset')
242
243     doc_ref = db.collection('hyper_searches').document(search_id)
244     try:
245         ground_dist_km = haversine_distance(sender['lat'], sender['
                lon'], recipient['lat'], recipient['lon'])
```

```
246         tle_urls_to_fetch = TLE_URLS_MEO_GEO if ground_dist_km >
               LEO_MAX_DISTANCE_KM else TLE_URLS_LEO
247         constellation_name = list(tle_urls_to_fetch.keys())[0]
248         tle_data = get_tle_data(tle_urls_to_fetch)
249
250         if tle_data:
251             satellites, name_map = parse_tle_string(tle_data)
252             t_search_start = datetime.fromisoformat(
                   t_search_start_iso)
253             current_time = t_search_start + timedelta(seconds=
                   t_offset)
254             t = timescale.from_datetime(current_time)
255
256             result = None
257             if mode == 'movable':
258                 result = find_movable_bounce_at_time(satellites,
                       name_map, sender, recipient, t)
259             else: # fixed
260                 plausible_sender_dirs, plausible_recipient_dirs =
                       set(), set()
261                 if recipient['lat'] > sender['lat']:
                       plausible_sender_dirs.add('N')
262                 if recipient['lat'] < sender['lat']:
                       plausible_sender_dirs.add('S')
263                 if recipient['lon'] > sender['lon']:
                       plausible_sender_dirs.add('E')
264                 if recipient['lon'] < sender['lon']:
                       plausible_sender_dirs.add('W')
265                 if not plausible_sender_dirs: plausible_sender_dirs
```

```
                = {'N','S','E','W'}
266             if sender['lat'] > recipient['lat']:
                    plausible_recipient_dirs.add('S')
267             if sender['lat'] < recipient['lat']:
                    plausible_recipient_dirs.add('N')
268             if sender['lon'] > recipient['lon']:
                    plausible_recipient_dirs.add('W')
269             if sender['lon'] < recipient['lon']:
                    plausible_recipient_dirs.add('E')
270             if not plausible_recipient_dirs:
                    plausible_recipient_dirs = {'N','S','E','W'}
271             plausible_paths = [(s_ant, r_ant) for s_ant in
                    ANTENNA_CONFIGS if s_ant['direction'] in
                    plausible_sender_dirs for r_ant in
                    ANTENNA_CONFIGS if r_ant['direction'] in
                    plausible_recipient_dirs]
272
273             result = find_fixed_bounce_at_time(satellites,
                    name_map, sender, recipient, t, plausible_paths)
274
275         if result:
276             result['constellation_searched'] =
                    constellation_name
277             @firestore.transactional
278             def update_in_transaction(transaction, doc_ref,
                    result):
279                 snapshot = doc_ref.get(transaction=transaction)
280                 if snapshot.get('status') == 'PENDING':
281                     transaction.update(doc_ref, {'results':
```

```
                                firestore.ArrayUnion([result]), 'status':
                                 'SUCCESS'})
282                 transaction = db.transaction()
283                 update_in_transaction(transaction, doc_ref, result)
284     except Exception as e:
285         print(f"Worker for timestamp offset {t_offset} failed: {e}")
286     finally:
287         @firestore.transactional
288         def increment_counter(transaction, doc_ref):
289             transaction.update(doc_ref, {'steps_completed':
                    firestore.Increment(1)})
290         transaction = db.transaction()
291         increment_counter(transaction, doc_ref)
292     return 'OK', 200
293
294 @app.route('/get-search-status/<search_id>') # Renamed to be generic
295 def get_hyper_search_status(search_id):
296     if db is None: return jsonify({'status': 'ERROR', 'message': '
           Database not configured'}), 500
297     doc_ref = db.collection('hyper_searches').document(search_id)
298     doc = doc_ref.get()
299     if not doc.exists: return jsonify({'status': 'INVALID_ID'}), 404
300     data = doc.to_dict()
301     if data.get('status') == 'SUCCESS':
302         return jsonify({'status': 'SUCCESS', 'result': data['results
               ']][0], 'mode': data.get('mode')})
303     completed = data.get('steps_completed', 0)
304     total = data.get('total_steps', 120)
305     if completed >= total:
```

```
306             duration = MOVABLE_SEARCH_DURATION_SEC if data.get('mode')
                    == 'movable' else FIXED_SEARCH_DURATION_SEC
307             return jsonify({'status': 'COMPLETE_NO_RESULT', '
                    search_duration_min': int(duration / 60)})
308         return jsonify({'status': 'PENDING', 'steps_completed':
                completed, 'total_paths': total})
309
310 @app.route('/_ah/warmup')
311 def warmup():
312     print("Warmup request received. Initializing instance and pre-
            caching TLE data.")
313     get_tle_data(TLE_URLS_LEO)
314     get_tle_data(TLE_URLS_MEO_GEO)
315     return '', 200, {}
```

1 Claims